1 Why an Operating System Manages Memory

Memory is a shared resource: the kernel and every task or process it runs all need it at the same time. With respect to memory, an operating system has four jobs.

- **Placement**: decide where the code, data, and stack of each program go, so that they do not overlap. Two programs that both use address `0x20000004` for their own data would destroy each other's values.
- **Sharing**: let programs use the same memory when they should, for example one copy of library code for every program, or a buffer handed to a device.
- **Protection**: stop a program from reading or writing memory that is not its own, whether by mistake (a bad pointer, a stack that grows too far) or on purpose.
- **More programs than memory**: when the programs together need more memory than there is, keep in RAM only the parts in use and store the rest elsewhere.

A flat firmware image, like the kernel built in the previous chapters, does only the first job, and only partly. The linker places every variable, but it places them file after file, with no border between the kernel's data and the data of each task.

```
20000004 B uptime_ms      kernel: millisecond counter
20000008 b tx_buf         kernel: UART transmit buffer
20000054 b a_lines        task A: printed lines
20000058 b b_seconds      task B: reported seconds
```

The output of `arm-none-eabi-nm -n` shows four variables packed together in `.bss`, just above `0x20000000`, a few bytes apart. Nothing marks `a_lines` as "task A only": if task B computed the address `0x20000054` and wrote there, it would change A's variable, and nobody would notice.

1.1 Static Versus Dynamic Allocation

Placement can be decided at two different times. With **static allocation**, every address and every size is fixed at link time by the linker script and the variable definitions; the map file shows the full layout of memory before the program runs, and no holes can appear because nothing is ever freed. The price is flexibility: a size cannot change at run time, and objects whose size is known only at run time cannot be created.

With **dynamic allocation**, memory is handed out at run time from a **heap**, through calls such as `malloc( n )` and `free( p )`. The layout then depends on the order of the calls, which is usually unpredictable. After many allocations and frees, the free memory is split into small holes between blocks still in use.

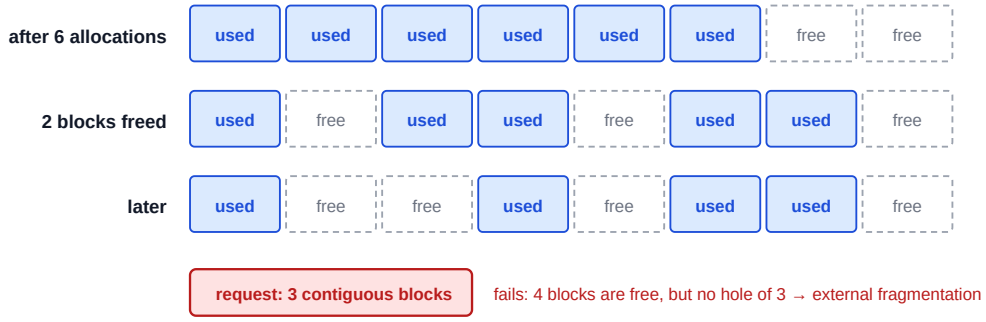


Figure 5.1: *External fragmentation. Four blocks are free, but no three of them are contiguous, so a request for three blocks fails.*

This is **external fragmentation**: enough free space exists, but in the wrong shape. Dynamic allocation has two more problems in an embedded system. It can fail at run time, when the heap is exhausted or too fragmented, and a device in the field usually has nobody to read the error. Its execution time is also not constant, because `malloc` may have to search many blocks, which is a problem for code with deadlines. For these reasons the kernel described here allocates everything statically.

1.2 The Memory-Management Spectrum

Memory-management schemes range from very simple to very capable. Two questions classify each of them: is the address used by the program **translated** into another address before it reaches memory, and does the hardware **protect** memory by stopping forbidden accesses?

Scheme	Translation	Protection	Where
Flat	No	No	Simple firmware: one address space for everybody
Static partitions	No	No	A fixed slice of RAM per task, decided at link time
MPU	No	Yes	Microcontrollers: hardware checks every access against a few regions
Paging (MMU)	Yes	Yes	Linux, Windows, macOS: each process has its own address space
Virtual memory	Yes	Yes	Paging plus a disk: programs can use more memory than the RAM

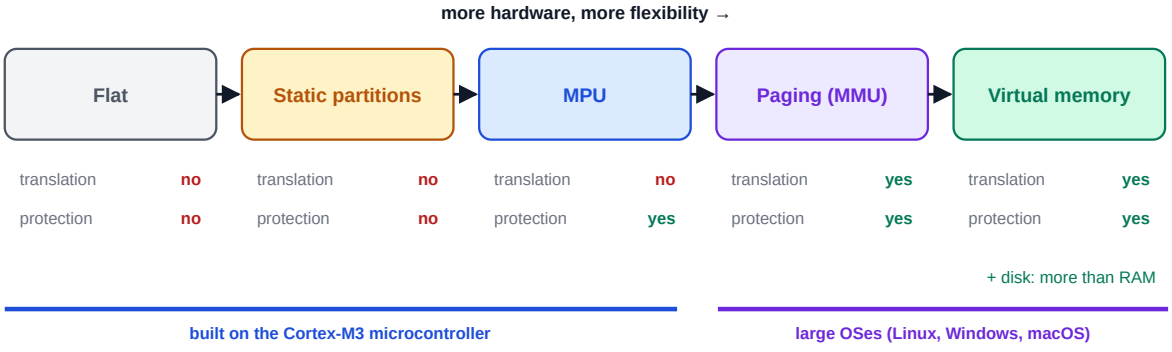


Figure 5.2: *The memory-management spectrum. Each step to the right needs more hardware and gives more flexibility.*

The first three steps can be built on a Cortex-M3 microcontroller; paging and virtual memory need an MMU and are described here as theory, because they explain what a microcontroller does differently.

1.3 Unprotected Memory: A Rogue Task

```
extern volatile uint32_t uptime_ms;     /* the kernel's counter */


printf( "B: writing to the kernel's uptime counter\n" );
uptime_ms = 0x12345678;                 /* a store into kernel memory */
```

Task B declares the kernel's counter with `extern`, so the linker gives it the real address, and then stores a value into it. The store succeeds: one second later B prints an uptime of 305 420 896 ms, because `0x12345678` is 305 419 896 and the kernel keeps counting from there. Task B has changed the kernel's clock for everyone.

Nothing stopped the write even though task B runs unprivileged. Unprivileged mode blocks only some instructions (such as `msr` to `CONTROL`) and access to the System Control Space; it does not check ordinary loads and stores to RAM. On the `mps2-an385` board the memories themselves have no notion of owner either: the bus answers every load and store from any code. A bug in one task can therefore silently break the kernel or another task, and the damage shows up far from the code that caused it. Stopping it requires hardware that checks every access, which is the job of an MMU or an MPU.

2 Paging and Virtual Memory

Large operating systems isolate processes with **paging**. A program uses **virtual addresses** (VA), the addresses in its pointers and instructions; the memory chips see **physical addresses** (PA), the addresses on the bus. Paging maps one to the other.

2.1 Pages, Frames, and the MMU

The virtual address space is divided into equal blocks called **pages**, often 4 KiB, and physical memory into **frames** of the same size. Any page can be placed in any frame. A **page table**, one per **process**, records which frame holds each page. Each entry also has a **valid** bit (is the page in a frame at all?) and permission bits: may the page be read, written, or executed, and may user code access it or only the kernel?

The **MMU** (Memory Management Unit) is the hardware that translates every address and checks its permissions on every load, store, and instruction fetch. This must be done in hardware, because doing it in software for every access would be far too slow. Only the **page number** is translated; the **offset** inside the page is copied unchanged, because a page and its frame have the same size. With 4 KiB pages the offset is the low 12 bits of the address ($2^{12} = 4096$).

2.2 Two-Level Page Tables

With 32-bit addresses and 4 KiB pages, the page number has 20 bits. A single flat table would need 2^{20} entries of 4 bytes, that is 4 MiB per process, even for a tiny program. Real systems therefore split the page number again and use two levels. A register called **PTBR** (page-table base register) holds the address of the current process's **page directory**; each directory entry points to a second-level page table.

```
0x00403007 = 0000 0000 0100 0000 0011 0000 0000 0111
           = 0000000001 | 0000000011 | 000000000111
             dir = 0x001  tbl = 0x003  offset = 0x007
```

The top 10 bits select the directory entry, the next 10 bits the table entry, and the low 12 bits are the offset. To translate `0x00403007`, the MMU reads directory entry 1 at `PTBR + 1 × 4`, which gives the table address, say `0x00021000`. It then reads entry 3 of that table at `0x00021000 + 3 × 4 = 0x0002100C`, which gives the frame, say `0x0009A000`, with the valid bit set. The physical address is the frame plus the offset: `0x0009A007`.

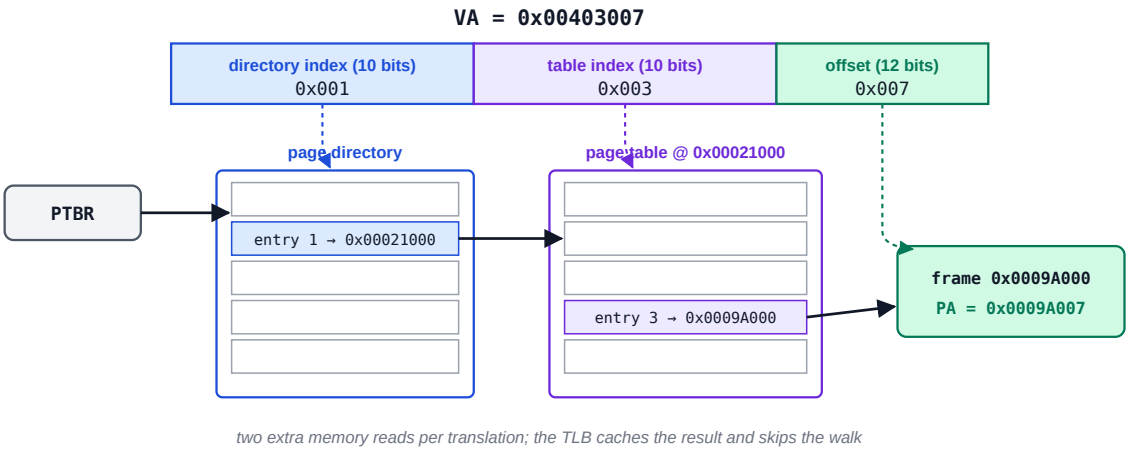


Figure 5.3: *Translation of 0x00403007 with a two-level page table. Each level costs one extra memory read.*

Two levels pay off because a second-level table exists only where memory is used. A small process needs one directory and a few tables instead of a flat 4 MiB table made almost entirely of invalid entries.

2.3 The TLB and the Effective Access Time

The walk costs two extra memory reads for every access the program makes, which would make memory three times slower. The **TLB** (Translation Lookaside Buffer) is a small, fast cache inside the MMU that keeps recent translations (page number → frame, with permissions). On a **TLB hit** the frame comes straight from the TLB; on a **miss** the MMU walks the tables and stores the result. When the OS changes a page table or switches to another process, it must **flush** the stale TLB entries, otherwise a process could reach the frames of the previous one.

The cost of paging is measured by the **effective access time** (EAT), the average time of one memory access including translation. With memory access time m, hit ratio h, and a two-level table, a hit costs m and a miss costs $3m$:

$$EAT = h \cdot m + (1 - h) \cdot 3m = m\,(3 - 2h)$$

The slowdown over plain memory is $2(1 - h)$: every 1% of misses makes memory 2% slower. With $m = 10$ ns, a hit ratio of 0.80 gives 14 ns (+40%), while 0.99 gives 10.2 ns (+2%). Processors with 64-bit addresses use four table levels, so a miss costs $5m$ and the slowdown doubles to $4(1 - h)$.

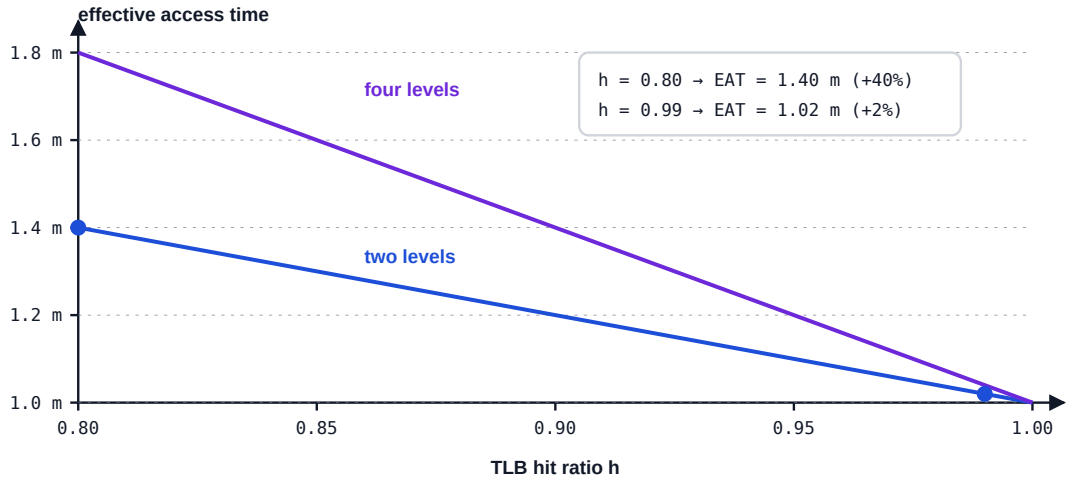


Figure 5.4: *Effective access time as a function of the TLB hit ratio, for two-level and four-level page tables.*

The hit ratio is high in practice because of **locality**: a loop that reads an array misses once on the first word of each 4 KiB page and then hits on the other 1023 words. The **TLB reach** (entries × page size, for example 64 × 4 KiB = 256 KiB) tells how much memory can be accessed without misses. After a flush the TLB is empty, so the same code runs slower right after a context switch. For a real-time task this variation is **jitter**: the time of an access depends on what ran before.

2.4 Page Faults

When the walk finds an entry whose valid bit is clear, or the access breaks the entry's permissions, the MMU raises a **page fault**:

1. the MMU stops the access and raises an exception, saving the faulting address;
2. the OS checks its records of the process's memory: if the address is not part of the program, or the access is not allowed, the program has a bug and the OS terminates it;
3. if the address is legal but the page is not in RAM, the OS finds a free frame, reads the page from disk, fills in the page-table entry, and sets the valid bit;
4. the handler returns and the hardware **restarts** the faulting instruction, which now succeeds.

The instruction is restarted, not skipped, because its load or store never happened. A page fault that reads from disk costs milliseconds, while a memory access costs nanoseconds. With page-fault rate p:

$$EAT = (1 - p) \cdot m + p \cdot t_{fault}$$

With $m = 200$ ns and $t_{fault} = 8$ ms, one fault every thousand accesses makes the machine about forty times slower, and keeping the slowdown under 10% requires fewer than one fault every 400 000 accesses. A single fault also stops the program for milliseconds at an unpredictable moment, which a real-time system cannot accept.

2.5 Virtual Memory

Paging combined with the disk is **virtual memory**. It gives **isolation**: each process has its own page table, so the same virtual address lands in different frames for different processes, and a process cannot even name another process's memory. It also gives **more memory than RAM**: pages not in use wait on disk and are brought back by page faults. Virtual memory therefore handles both protection and the fourth job of the OS, but it needs an MMU, page tables in RAM, and a disk.

A microcontroller has none of these, and it has good reasons not to: an MMU costs silicon area and energy, TLB misses make timing unpredictable, and there is no disk to page to. The Cortex-M3 offers instead an optional **MPU**, which checks addresses against a few regions without translating them. Its regions, permissions, and registers are described in the MPU section of the chapter on the ARM Cortex-M3 architecture. The rest of this chapter builds protection with it, in two steps: first static partitions, which give each task a block of memory with the shape the MPU needs, then the MPU itself.

3 Static Task Partitions

A **partition** is a slice of RAM that belongs to a single task and holds only that task's stack and variables.

3.1 Partition Rules

The partitions follow five rules.

- **They are fixed**: the linker chooses them at link time (static allocation), and no task is created, grown, or moved at run time.
- **Each one is 4 KB and aligned to 4 KB**: the size is a power of two and the start address a multiple of it, such as `0x20000000` or `0x20001000`. This is exactly the shape of an MPU region.
- **They cover RAM only**: code, constants, and string literals stay in FLASH, shared by the kernel and all tasks. Both tasks call the same `printf`, and one copy of its code is enough.
- **The stack is at the bottom**: the stack takes the lowest 1 KB, and the task's variables come right above it.
- **The kernel keeps the rest**: the kernel's `.data` and `.bss` come after the partitions, and the kernel stack stays at the top of RAM.

The position of the stack matters. The stack grows down, so a stack that grows too far leaves the partition below its start instead of overwriting the task's own variables. With the MPU enabled, that first out-of-partition store causes a fault; with the stack at the top, an overflow would silently overwrite the task's variables, and no protection hardware could tell that write from a legitimate one.

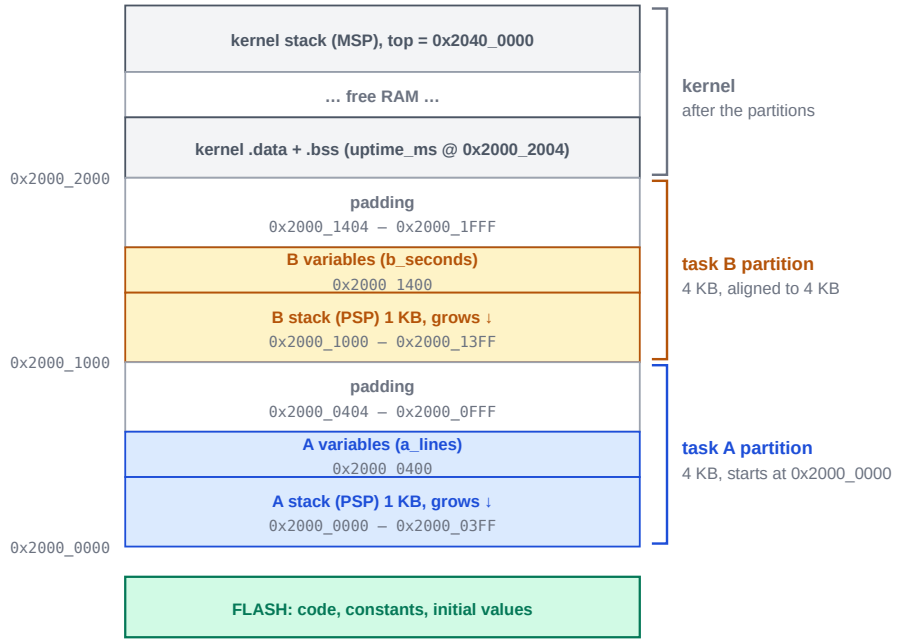


Figure 5.5: *RAM layout with two task partitions. Each 4 KB partition holds a 1 KB stack at the bottom, the task's variables, and padding.*

Power-of-two sizes have a cost: a task that needs 1.1 KB still gets 4 KB. On a microcontroller with a few hundred KiB of RAM and a few tasks, this waste is usually acceptable.

3.2 Linker Script Constructs for Partitions

```
.task_a_data : { ... } > RAM AT > FLASH       /* runs in RAM, initial bytes stored in FLASH */
_task_a_data_load = LOADADDR(.task_a_data);  /* the FLASH address of those bytes           */
.task_a_stack ALIGN(ORIGIN(RAM), 4096) (NOLOAD) : { ... } > RAM
*task_a.o(.data .data.* .bss .bss.* COMMON)  /* these input sections, from task_a.o only */
```

The partitions reuse the linker-script constructs of the boot chapter and add placement by file.

Construct	Meaning
`> RAM AT > FLASH`	The section runs in RAM (VMA) but its initial bytes are stored in FLASH (LMA), as for `.data`
`LOADADDR(section)`	The load address of a section, here the FLASH address of the task's initial values
`(NOLOAD)`	Reserves addresses but stores nothing in the image: right for stacks and padding
`ALIGN(x, 4096)`	`x` rounded up to a multiple of 4096; written before the colon, it sets the section start
`. = start + 4096`	Moves the location counter to the end of the partition
`*task_a.o(...)`	Takes the listed input sections from the object file `task_a.o` only
`ASSERT(cond, "msg")`	Makes the link fail if `cond` is false

A file-name pattern in front of the parentheses selects input sections from one object file only, so every variable written in `src/task_a.c` goes to A's partition without any special marking in the source. Three details make the rules work. First, **the first matching rule wins**: the generic `*(.data*)` and `*(.bss*)` rules of the kernel would also match `task_a.o`, so the partition rules must come before them in the script. Second, the task's `.bss` is placed inside the loaded section together with `.data`: its zeros are then stored in FLASH (a few bytes), and a single copy loop initialises every task variable. Third, the first partition must use `ALIGN(ORIGIN(RAM), 4096)`, because right after `.text` the location counter still holds a FLASH address.

3.3 The Partition Block

```
    /* Task A partition, 4 KB: [ stack 1 KB | A's .data + .bss | padding ] */
    .task_a_stack ALIGN(ORIGIN(RAM), 4096) (NOLOAD) : {
        _task_a_part_start = .;            /* partition start = stack bottom */
        . = . + TASK_STACK_SIZE;           /* reserve the 1 KB stack */
        _task_a_stack_top = .;             /* the stack grows down from here */
    } > RAM
    .task_a_data : {
        . = ALIGN(4);
        _task_a_data_start = .;            /* run address (RAM) */
        *task_a.o(.data .data.* .bss .bss.* COMMON)   /* all of task_a.o's variables */
```

```
        . = ALIGN(4);
        _task_a_data_end = .;
    } > RAM AT > FLASH                      /* runs in RAM, image stored in FLASH */
    _task_a_data_load = LOADADDR(.task_a_data);   /* FLASH address of the image */
    .task_a_pad (NOLOAD) : {
        . = _task_a_part_start + PART_SIZE;   /* jump to the partition end */
        _task_a_part_end = .;
    } > RAM

    /* Task B partition: the same three sections, in the next 4 KB. */
    .task_b_stack ALIGN(4096) (NOLOAD) : { ... } > RAM
    .task_b_data : { ... } > RAM AT > FLASH
    .task_b_pad (NOLOAD) : { ... } > RAM

    /* Link-time guards: each task's variables must fit in its 4 KB. */
    ASSERT(_task_a_data_end <= _task_a_part_end, "task A data does not fit in its partition")
    ASSERT(_task_b_data_end <= _task_b_part_end, "task B data does not fit in its partition")
```

Each partition is made of three output sections, placed before the kernel's `.data` and `.bss`, with `PART_SIZE = 0x1000` and `TASK_STACK_SIZE = 0x400`. `.task_a_stack` starts at the first 4 KB boundary at or above `0x20000000`, marks the start of the partition, reserves 1 KB, and marks the stack top at `0x20000400`; it is `NOLOAD` because a stack has no initial content. `.task_a_data` follows immediately and collects every variable of `task_a.o`; the two `ALIGN(4)` keep both ends on a word boundary, because the copy loop copies whole words. `.task_a_pad` stores nothing and only moves the location counter to the end of the partition, so the next partition starts after A's full 4 KB. Task B repeats the three sections, with `ALIGN(4096)` alone because the location counter is already in RAM. The two `ASSERT` lines catch a task that grows too big at build time rather than at run time.

3.4 Checking the Layout

```
00000dd0 A _task_a_data_load
20000000 B _task_a_part_start
20000400 B _task_a_stack_top
20000400 d a_lines
20001000 B _task_a_part_end
20001000 B _task_b_part_start
20001400 B _task_b_stack_top
20001400 d b_seconds
20002000 B _task_b_part_end
20002004 B uptime_ms
```

The `nm -n` output confirms the layout: task A owns `0x20000000` – `0x20001000` and task B `0x20001000` – `0x20002000`, both 4 KB and 4 KB-aligned, and the kernel's `uptime_ms` has moved after the partitions, to `0x20002004`. `a_lines` and `_task_a_stack_top` have the same address, `0x20000400`, but they do not overlap. The stack top is an exclusive bound: the Cortex-M stack is full descending, so with `sp = 0x20000400` the first

push writes at `0x200003FC`, and the word at `0x20000400` is free for the first variable. The map file shows the same layout from the linker's point of view, with a FLASH load address for each `.task_X_data` section and none for the `NOLOAD` stacks.

3.5 Initialising Partition Variables

```
void partition_init( const struct task *t )
{
    const uint32_t *src = t->data_load;     /* initial values, in FLASH */
    uint32_t *dst = t->data_start;          /* the variables, in RAM */
    while ( dst < t->data_end )
        *dst++ = *src++;                    /* copy one word, move both on */
}
```

The variables of a task run in RAM but start in FLASH, so the kernel must copy them before the task runs for the first time. `partition_init` is the same loop that `Reset_Handler` runs for the kernel's `.data`, executed once per task with the addresses stored in its `struct task`, which the kernel fills from the linker symbols. No separate zeroing loop is needed, because the task's `.bss` is part of the loaded section.

3.6 Partitions Arrange but Do Not Enforce

Running the rogue task again after the change gives exactly the same result as before: task B still corrupts `uptime_ms`, now at `0x20002004`. The linker decides only *where* things are; at run time a store to `0x20002004` is an ordinary store, and nothing compares it with B's partition. What has changed is the shape of memory: each task's memory is now a single aligned 4 KB block, so one hardware rule per task is enough to describe what that task may touch.

4 Starting and Ending Tasks

A task runs in unprivileged Thread mode on its own stack. Starting and ending such a task requires the two stack pointers of the Cortex-M3 and the exception return mechanism, both described in the chapter on the ARM Cortex-M3 architecture.

4.1 Kernel Stack and Task Stacks

Each task's stack is a **PSP** stack inside its own partition, while the kernel and every exception handler use the **MSP** stack at the top of RAM. A running task has `CONTROL = 3`: `nPRIV = 1` (unprivileged) and `SPSEL = 1` (PSP). The stack must lie inside the partition because it holds the task's local variables, return addresses, and the exception frame that the hardware pushes on every interrupt; with the stack inside the partition, the MPU can restrict the task to its own 4 KB. Because Handler mode always uses MSP, the kernel's own work never runs on a task's stack.

4.2 Entering a Task with a Fake Exception Frame

The kernel cannot start a task with a function call. When it starts a task it is running inside the SVC handler, in Handler mode, privileged, and on MSP; a call would keep that mode, privilege, and stack, and the `svc` would never end. The only hardware mechanism that changes mode, privilege, and stack in a single step is the **exception return**. The kernel therefore **fakes an interruption**: it writes on the task's stack the frame that the hardware would have pushed if the task had been interrupted just before its first instruction, points PSP at it, and returns from the `svc` with `EXC_RETURN = 0xFFFFFFFD` (Thread mode, PSP).

Word	Address (task A)	Register	Value written
`frame[0..4]`	`0x200003E0` – `0x200003F3`	`r0` – `r3`, `r12`	0
`frame[5]`	`0x200003F4`	`lr`	`task_exit`
`frame[6]`	`0x200003F8`	`pc`	`t->entry & ~1`, the address of `task_a`
`frame[7]`	`0x200003FC`	`xPSR`	`0x01000000`, Thumb bit only

```
__asm volatile (
    "msr psp, %0         \n" /* PSP = frame */
    "mrs r1, control     \n" /* r1 = CONTROL */
    "orr r1, r1, #1      \n" /* r1 |= 1: set nPRIV */
    "msr control, r1     \n" /* CONTROL = r1 */
    "isb                 \n" /* make the new CONTROL take effect now */
    : : "r" ( frame ) : "r1", "memory" );
return EXC_RETURN_THREAD_PSP;              /* 0xFFFFFFFD */
```

`task_launch` first records the current task, sets the MPU region for its partition, and copies its variables with `partition_init`. It then writes the 8-word frame at `t->stack_top - 8` and runs the assembly block, which is needed because C cannot name PSP or `CONTROL`. The returned value travels back to `SVC_Handler`, which ends the exception with it. The hardware pops the 8 words from PSP and continues at the popped `pc`, `task_a`, in

Thread mode, unprivileged, on PSP.

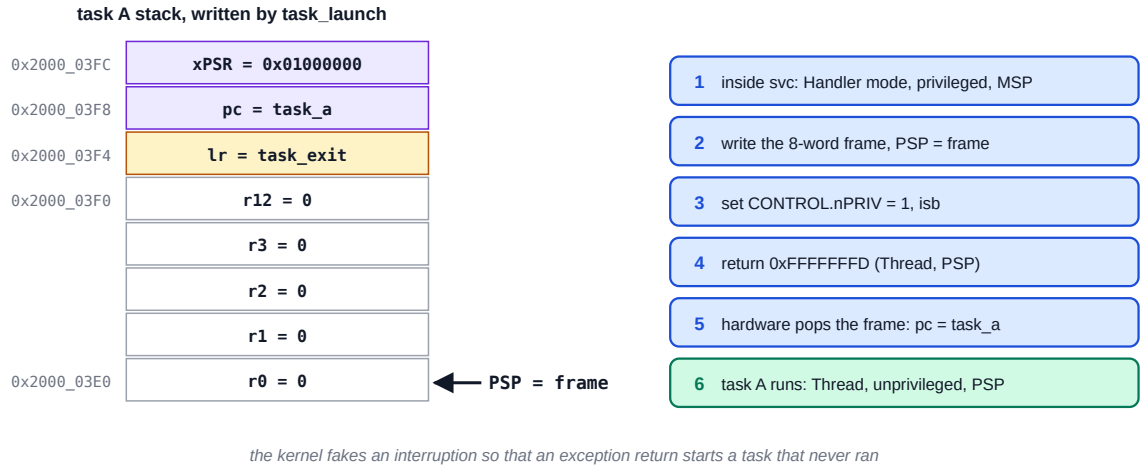


Figure 5.6: *Starting a task. The kernel writes a fake exception frame on the task's stack, and the exception return "resumes" a task that has never run.*

Several details of the frame are essential. Setting `nPRIV` inside the handler has no effect until the return, because Handler mode is always privileged; after the return `CONTROL` is 3, since the exception return with `0xFFFFFFFD` sets `SPSEL`. The `xPSR` word must have bit 24 (the Thumb bit) set, otherwise the first instruction raises a UsageFault (`INVSTATE`). The `pc` word has bit 0 cleared, because it must be a real instruction address rather than a code pointer. Finally, `lr` holds `task_exit` , so a task that simply returns from its function lands in the exit path.

4.3 Choosing the Stack in SVC_Handler

```
    tst lr, #4                  /* flags = lr & 4: is bit 2 set? */
    ite eq                      /* if ((lr & 4) == 0) ... else ... */
    mrseq r0, msp               /*     frame = MSP;  // came from main() */
    mrsne r0, psp               /* else frame = PSP;  // came from a task */
    mov r1, lr                  /* exc_return = lr: second argument */
    bl svc_dispatch             /* r0 = svc_dispatch(frame, exc_return) */
    bx r0                       /* return_from_exception(r0) */
```

With tasks, `svc` can arrive from two places: from a task, on PSP, and once from `main` , on MSP. Bit 2 of `EXC_RETURN` tells which stack holds the stacked frame. `tst lr, #4` sets the Z flag when the bit is 0, and `ite eq` (if-then-else) makes exactly one of the two following `mrs` instructions execute. The entry then calls `svc_dispatch` with the frame and the `EXC_RETURN` value, copying `lr` first because `bl` overwrites it, and ends the exception with the value the dispatcher returns. In pseudo-C: `frame = (lr & 4) ? PSP : MSP; lr = svc_dispatch(frame, lr); return_from_exception(lr);` .

Returning a value is what lets a system call return *somewhere else*. Ordinary services (`SYS_WRITE` , `SYS_READ` , `SYS_UPTIME`) write their result into `frame[0]` and return `exc_return` unchanged, so the exception goes back to the caller. `SYS_START_TASKS` , issued once by `main` with `svc #5` , returns into the first task; `SYS_EXIT` returns into the next task or back to `main` . Each `svc` still ends at once: the kernel never waits inside a handler.

4.4 Ending a Task

```
sys_exit:
    svc #4                      /* svc(SYS_EXIT) */
    b sys_exit                  /* never reached; if it were, ask again */

task_exit:
    bl sys_exit                 /* sys_exit(); it does not come back */
    b task_exit                 /* never reached; if it were, try again */
```

A task ends either by calling `sys_exit()` or by returning from its function, in which case it lands in `task_exit`, the address placed in `frame[5]`. On the kernel side, `SYS_EXIT` from a task calls `task_exit_current`:

```
uint32_t task_exit_current( void )
{
    kprintf( "task %s exited\n", tasks[current].name );
    if ( current + 1 < ( int ) NUM_TASKS )
        return task_launch( current + 1 );      /* start the next task */
    current = -1;                               /* no task runs any more */
    /* asm block: CONTROL = CONTROL & ~1;  clear nPRIV, then isb */
    return EXC_RETURN_THREAD_MSP;               /* 0xFFFFFFF9: back to main() */
}
```

When task A ends, the function launches task B: it moves the MPU region to B's partition and returns B's fake frame with `0xFFFFFFFD`, so the exception returns straight into B. When the last task ends, the kernel clears `nPRIV` and returns `0xFFFFFFF9` (Thread mode, MSP), so `main` continues after its `svc #5`. Clearing `nPRIV` is essential: `main` runs in Thread mode, and with `nPRIV` still at 1 it would run unprivileged and fault on its first access to the UART. It could not fix this by itself, because unprivileged code cannot write `CONTROL`.

5 Protecting Tasks with the MPU

With the partitions in place, two MPU regions are enough to describe what the running task may touch. The region encoding (`RBAR` , `RASR` , `SIZE` , `AP` , `XN`) is described in the MPU section of the Cortex-M3 chapter.

5.1 Two Regions per Task

```
void mpu_init( void )
{
    /* Region 0 = FLASH: base 0, 4 MB = 2^22 B, read-only, code may run. */
    MPU->RBAR = 0x00000000 | MPU_RBAR_VALID | 0;          /* = 0x00000010 */
    MPU->RASR = MPU_RASR_AP( MPU_AP_RO ) | MPU_RASR_C
              | MPU_RASR_SIZE( 22 ) | MPU_RASR_ENABLE;    /* = 0x0602002B */
    /* MPU on. PRIVDEFENA: the kernel keeps the default memory map. */
    MPU->CTRL = MPU_CTRL_ENABLE | MPU_CTRL_PRIVDEFENA;     /* = 5 */
    __asm volatile ( "dsb" ::: "memory" );
    __asm volatile ( "isb" ::: "memory" );
}

void mpu_set_task_region( uint32_t base, uint32_t size )
{
    uint32_t log2_size = __builtin_ctz( size );           /* 4096 -> 12 */
    /* Region 1 = the partition: read-write, never executable (XN). */
    MPU->RBAR = base | MPU_RBAR_VALID | 1;                 /* B: 0x20001011 */
    MPU->RASR = MPU_RASR_XN | MPU_RASR_AP( MPU_AP_FULL_RW )
              | MPU_RASR_S | MPU_RASR_C
              | MPU_RASR_SIZE( log2_size ) | MPU_RASR_ENABLE; /* 0x13060017 */
    __asm volatile ( "dsb" ::: "memory" );
    __asm volatile ( "isb" ::: "memory" );
}
```

`mpu_init` runs once, before the first task starts. It makes **region 0** cover the 4 MB of FLASH as read-only and executable for everyone, and enables the MPU with `PRIVDEFENA` , so that privileged code (the kernel, `main` , and every handler) keeps the default memory map. `mpu_set_task_region` is called by `task_launch` for every task. It points **region 1** at the partition of the task about to run, as read/write and execute-never. `__builtin_ctz` (count trailing zeros) computes $\log_2$ of the power-of-two size, and the `SIZE` macro applies the encoding. The `dsb` and `isb` barriers make sure that no later access is checked against the old regions.

The "MPU state" of a task is therefore a single value, the base of region 1. Everything else (the other task's partition, kernel data, kernel stack, and peripherals) has no region and is forbidden to unprivileged code. A correct program prints exactly the same lines as before: its code and constants are in FLASH, its variables, stack, and the `printf` buffer (a local variable) are in its partition, and every access to the UART happens inside the privileged kernel through system calls.

5.2 MemManage Reports

```
uint32_t cfsr = SCB->CFSR;
kprintf( "MemManage: CFSR=0x%x\n", cfsr );
if ( cfsr & SCB_CFSR_MMARVALID )                /* MMFAR is valid only then */
    kprintf( "  address 0x%x\n", SCB->MMFAR );
if ( cfsr & SCB_CFSR_DACCVIOL )
    kprintf( "  cause: data access\n" );
if ( cfsr & SCB_CFSR_IACCVIOL )
    kprintf( "  cause: instruction fetch\n" );
if ( cfsr & SCB_CFSR_MSTKERR )
    kprintf( "  cause: stacking\n" );
```

`MemManage_Handler` runs privileged and on MSP, so it can always print. It reads `CFSR` once, so that every test sees the same value, prints `MMFAR` only when `MMARVALID` is set, and tests each cause separately, because several bits can be set at the same time. With the MPU enabled, the rogue store never happens: the report reads `CFSR = 0x82` (`MMARVALID` + `DACCVIOL`) with address `0x20002004`, which `arm-none-eabi-nm` identifies as `uptime_ms`. The fault points at the exact instruction and variable involved, instead of letting the damage show up later and elsewhere.

A **stack overflow** is caught in the same way. A recursion that needs about 4 KB of stack in task B, which has only 1 KB, produces `CFSR = 0x92` with address `0x20000FDC`: `0x80 + 0x10 + 0x02`, that is `MMARVALID` + `MSTKERR` + `DACCVIOL`. The address is 36 bytes below B's partition, inside A's partition, which has no region while B runs. The stacking cause appears because the CPU also tried to push the exception frame on the overflowing PSP; the frame is lost, but the handler runs on MSP and its report is still correct. Without the MPU, the same program keeps corrupting memory and ends much later with a misleading `UsageFault` (`INVSTATE`), far from the real cause.

5.3 Why the Kernel Must Check Pointers

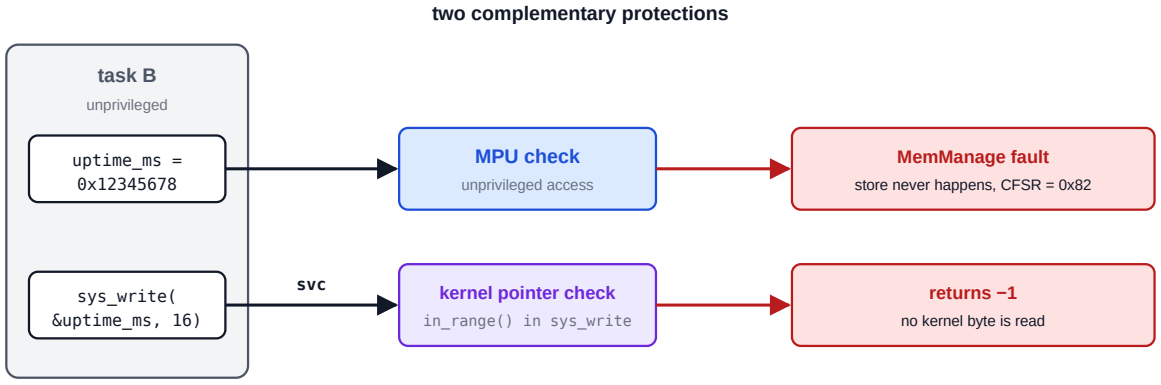
```
int r = sys_write( (const char *) &uptime_ms, 16 );
```

The MPU stops what a task does by itself, but not what a task asks the kernel to do. A task that passes a kernel address to `sys_write` makes the kernel read that memory on its behalf: the handler runs privileged, `PRIVDEFENA` gives it the whole map, and the call prints 16 bytes of kernel data (the counter and the start of the UART buffer). The kernel must therefore check every pointer it receives against what *that task* could access by itself.

```
static int in_range( uint32_t p, uint32_t len, uint32_t start, uint32_t end )
{
    return p >= start && p <= end && len <= end - p;
}
```

```
/* in sys_write_impl: accept the buffer only inside the caller's
 * partition or inside FLASH; the kernel itself (no task) is trusted. */
const struct task *t = task_current();
if ( t != 0
     && !in_range( a0, len, t->part_start, t->part_end )
     && !in_range( a0, len, 0x00000000, 0x00400000 ) )
    return ( uint32_t ) -1;
```

`in_range` checks that `[p, p + len)` lies entirely inside `[start, end)`. The obvious test `p + len <= end` is wrong, because `p + len` can wrap around 2^{32}: with `p = 0x20001F00` and `len = 0xFFFFFF00`, the sum wraps to `0x20001E00` and the check would pass, letting the kernel read almost all of memory. Once `p <= end` is known, `end - p` cannot wrap, so `len <= end - p` is always correct. `sys_write_impl` accepts a buffer only if it lies entirely in the caller's partition or entirely in FLASH, where string constants live; otherwise it returns −1 before reading anything. The task receives an error it can handle, not a fault, and the task's own `printf` still works because it formats its text in a buffer on the task's stack.



the MPU stops what a task does itself; the kernel stops what a task asks the privileged kernel to do

Figure 5.7: *Two complementary protections. The MPU stops direct accesses by unprivileged code; the kernel's pointer checks stop indirect accesses made through system calls.*